

0.87.3. Agrandir une lettre : `texteGrand` — un mot

Un mot entier agrandi : `texteGrand(3, 7, "JEU", 4)`. Chaque lettre prend 4×4 cases ; le mot, 12×4 .

1. Ce qu'il faut comprendre

Pas seulement une lettre : `texteGrand` agrandit un **mot entier**. Les lettres se suivent, chacune agrandie.

La largeur du mot : 3 lettres $\times$ 4 cases = **12 cases**. Il faut qu'elle tienne dans les 20 colonnes de l'écran :

"BONJOUR" (7 lettres) tient à la taille 2 (14 cases), pas à la taille 3 (21 cases).

Un titre de jeu, c'est exactement ça : un mot en grand, au milieu de l'écran.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texteGrand>` → [0.86.3. La fonction `texteGrand\(\)` — un texte agrandi](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1258 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          55 o ← la console, à l'allumage — la mise en route de la console
texte_grand_0()    195 o ← main() — ta fonction
main()             18 o ← démarrage — ta fonction
VBlank             8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage      28 o ← image() — attend l'image suivante
AttendreVBlank     16 o ← démarrage — attend que l'écran soit libre
AdresseCase        10 o ← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles       18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage — vide les deux cartes de l'écran
Tuiles            848 o ← CopierTuiles — la police entière, et 9 tuiles dessinées
GRAND_TABLE_0_0_0  12 o ← poser() — une table gravée
GRAND_TABLE_0_1_1  12 o ← poser() — une table gravée
GRAND_TABLE_0_2_2  12 o ← poser() — une table gravée
GRAND_TABLE_0_3_3  12 o ← poser() — une table gravée
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()            × 4   122 o sur place, dans texte_grand_0() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o),
GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
image()            × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
texte_grand_0() × 1    3 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

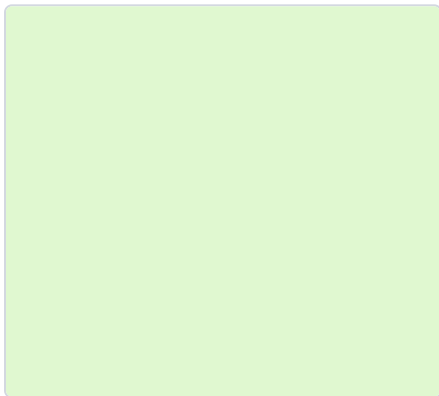
```
#include <texteGrand> // écrit un texte agrandi, de 1 à 20 fois
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

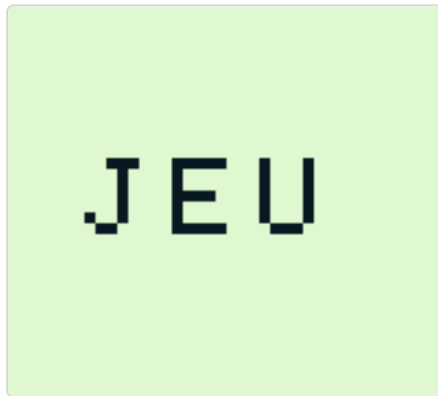
```
int main() {  
    // 3 lettres × 4 cases = 12 cases de large, 4 de haut : de (3, 7) à (14, 10).  
    texteGrand(3, 7, "JEU", 4);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le mot JEU en grandes lettres, au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le J, le E et le U, chacun dans ses 4 × 4 cases

✓ rien après le mot

Ensuite : **0.87.4. Agrandir une lettre : `texteGrand` — vingt fois** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++